

# Online Event Management System

<https://github.com/aiyshawary/Online-Event-Management-System>

[SQL](#)   [NoSQL](#)   [Database Design](#)   [Normalization](#)

## OVERVIEW

---

A database-driven system for managing events end-to-end, allowing customers to register, browse and book events with discounts, while giving administrators full control over event listings, users, payments and preferences.

## IMPACT

---

Designed and implemented a normalized relational database and NoSQL schema for an Online Event Management System, covering customer registration, booking workflows, discount management, and payment processing.

## TECHNICAL WALKTHROUGH

---

Online Event Management System – Project Explanation This project is a database-driven Online Event Management System designed to help users register, browse, and book events online, while allowing admins to manage events, users, discounts, and payments efficiently.

### Problem it Solves

**Managing events manually is time-consuming and error-prone—especially when handling**

**Multiple event types**

**User registrations**

**Discounts and special preferences**

**Payments and bookings**

This system provides a centralized, structured database that automates all these processes.

### Key Actors

#### User

**Registers using email and password**

Browses events (Dance, Music, Arts, Sports, etc.)

**Books events with number of seats**

**Applies discounts and special preferences**

**Makes payment using multiple modes**

**Can update or cancel bookings**

#### Admin

**Logs in with admin credentials**

**Adds, updates, or deletes:**

**Events**

**Discounts**

## **Special preferences**

### **User details**

#### **Manages pricing and discount rates**

Core Database Modules (Entities)

### **User**

#### **Stores registered user details:**

##### **User ID, Email, Username, Password**

Users must register before booking any event.

### **Event Details**

#### **Stores complete event information:**

##### **Event name, type, date & time**

##### **Duration and base price**

Each event belongs to a specific event category.

### **Event Discount**

#### **Manages discount options:**

##### **Premium account**

##### **Group booking**

##### **Coupon-based discounts**

Each discount has a rate reduced value.

### **Special Preference**

#### **Optional add-ons for users:**

##### **VIP seats**

##### **Backstage access**

##### **Exclusive services**

These come with extra charges.

### **User Booking**

#### **Acts as the core linking table:**

#### **Connects User, Event, Discount, and Special Preference**

##### **Stores number of seats booked**

Each booking has a unique Booking ID (BID) This table enforces referential integrity using foreign keys.

Amount (Payment)

#### **Handles payment details:**

##### **Payment ID**

##### **Booking ID**

Mode of payment (Credit, Debit, Wallet, etc.)

### **Admin**

Stores admin login credentials and permissions.

## Database Design Highlights

Fully normalized (3NF) to avoid redundancy

**Strong foreign key relationships**

**Uses indexes for faster search**

**Supports views for business insights**

## Important SQL Features Used

### Queries

Simple queries (filter, sort) Aggregate queries (COUNT, SUM, AVG) Join queries (INNER, LEFT, RIGHT joins)

### Views

#### Examples:

Total amount payable by each user (including discounts & extra charges)

**Users who registered but never booked**

**Number of bookings per user**

**Events sorted by price for easy searching**

### Indexes

**Faster event search by type & name**

**Unique email constraint for users**

### Faster lookup for bookings, discounts, and prices

Overall Summary (One-Line) This project implements a relational database-based Online Event Management System that efficiently handles user registrations, event bookings, discounts, special preferences, and payments with strong data integrity and optimized query performance.

## KEY DETAILS

---

- Designed a fully normalized relational schema (up to 3NF) with entities for Users, Events, Bookings, Payments, and Discounts.
- Modeled equivalent NoSQL document structures for flexible event-type storage and preference handling.
- Admin interface supports CRUD operations on events, user management, and discount configuration.
- Customer portal enables account creation, event browsing, booking with preference selection, and payment tracking.
- Schema documented with ER diagrams, normalization steps, and a detailed project report.